# OWASP Top 10:2025 Security Assessment Report

## Epotech Cybersecurity Platform

| | |
|---|---|
| **Application Name** | Epotech Cybersecurity Platform |
| **Organization** | Epotech |
| **Report Title** | OWASP Top 10:2025 Web Application Security Assessment |
| **Assessment Date** | 2026-10-29 |
| **Report Version** | 1.0 |
| **Confidentiality Classification** | **CONFIDENTIAL — Internal Use Only** |
| **Assessor** | Antigravity AI Security Assessment Engine |
| **Environment Assessed** | Local / Development (source code static review + configuration analysis) |

> **CAUTION**
> This report contains sensitive security findings. Distribute only to authorized personnel. Do not share with unauthenticated parties or publish in public repositories.

## SECTION 1 — EXECUTIVE SUMMARY

### Assessment Objectives

This assessment was commissioned by the authorized owner of the Epotech Cybersecurity Platform to perform a comprehensive, evidence-based security review aligned with the **OWASP Top 10:2025** vulnerability classification. The assessment was conducted by performing static source code analysis, configuration review, dependency analysis, and manual security logic review of all accessible project artifacts.

### Scope

The assessed application is a React (Vite) single-page application serving as a cybersecurity services portal. It connects to a Supabase (PostgreSQL) backend via the Supabase JS SDK and references a FastAPI Python serverless backend hosted on Vercel. The application manages client service engagement requests and provides a security scanning interface.

### Overall Results

**6 confirmed security findings** were identified across 5 OWASP Top 10:2025 categories. **3 findings are rated CRITICAL or HIGH severity** and require immediate remediation.

| Severity | Count |
|---|---|
| CRITICAL | 2 |
| HIGH | 1 |
| MEDIUM | 2 |
| LOW | 1 |
| INFORMATIONAL | 2 |

### Principal Business Risks

1. **Any attacker can escalate to Admin** by registering with an email containing the string `admin` — granting full platform access, the ability to view all client data, delete records, and trigger scans against arbitrary targets.
2. **Production credentials are committed to the repository** — the live Supabase Project URL, Anon Key, and a Vercel OIDC token are present in tracked files ( `.env` , `.env.local` ), creating a permanent secret exposure risk.
3. **The RLS (Row Level Security) policy allows unauthenticated (anon) users to read, insert, and update all service requests** — any internet user can exfiltrate client data without authentication.
4. **A hardcoded demo session bypass completely circumvents authentication** — if the Supabase network call fails for any reason (including an attacker-induced fault), the system falls back to granting a session based solely on the email the user typed.

### Highest-Priority Remediation Actions

1. Immediately rotate the exposed Supabase Anon Key and Vercel OIDC token.
2. Remove the email-based admin role inference in `auth.js` and replace with server-side role verification from Supabase `user_metadata` or a dedicated roles table.
3. Remove the demo session bypass / offline fallback login path.
4. Harden all RLS policies to enforce per-user data isolation.
5. Move all credentials from `.env` into a secret manager; ensure `.env` and `.env.local` are never committed.

## SECTION 2 — SCOPE AND AUTHORIZATION

| | |
|---|---|
| **Authorized Project Directory** | `c:\Users\hngga\Downloads\epoteck new` |
| **Application URL** | Not publicly provided; deployment target is Vercel |
| **Environment Assessed** | Local/Development source code |
| **Components Included** | All frontend source files, configuration files, schema SQL, CI/CD workflow, dependency manifests |
| **Excluded Systems** | Live Supabase production database (no queries executed), Vercel production infrastructure, third-party CDN/font providers |
| **Test Accounts Used** | None — static review only; no live traffic generated |
| **Testing Dates** | 2026-10-29 |
| **Scope Limitations** | No live application URL was provided. Dynamic runtime tests (active HTTP probing, session manipulation, live API testing) were not executed. The Python FastAPI backend ( `/api/index.py` ) was not found in the project directory and could not be reviewed. |

## SECTION 3 — APPLICATION ARCHITECTURE

### Frontend

- **Framework:** React 18.3 with Vite 6.1 (SPA — Single Page Application)
- **Routing:** React Router DOM v7 (client-side routing)
- **Styling:** Tailwind CSS v4
- **UI Libraries:** Framer Motion, Lucide React, React Slick
- **Deployment:** Vercel (static build + serverless functions)

### Backend & Database

- **Backend Type:** Serverless Python (FastAPI, referenced in `vercel.json` as `/api/index.py`). *Note: The FastAPI source file was **not found** in the project directory during assessment. All backend endpoints are referenced but unverifiable.*
- **API Routes Referenced:** `/api/health`, `/api/requests`, `/api/requests/:id`, `/api/audit/scan`, `/api/scan/comprehensive`
- **Database Provider:** Supabase (PostgreSQL hosted)
- **Project URL:** `https://rwfbcsedjgpesnbfeprd.supabase.co` (REDACTED in production reporting; present in committed `.env`)
- **Tables:** `service_requests`, `security_audits`
- **Row Level Security:** Enabled, but configured with overly permissive policies

### Authentication, Authorization & Integrations

- **Auth Provider:** Supabase Auth (`supabase.auth.signInWithPassword`)
- **Session Storage:** `localStorage` (key: `epotech_user`)
- **Role Determination / Guards:** Client-side email string matching in `auth.js`; `ProtectedRoute` React component.
- **Demo/Bypass Mode:** Present; grants sessions without Supabase verification on network failure.
- **External Integrations:** Supabase (database+auth+realtime), Vercel (hosting), Sentry (monitoring), Google Fonts.

### Trust Boundaries

```
[User Browser]
    ↓ localStorage session
[React SPA — client-side role enforcement]
    ↓ Supabase JS SDK (anon key embedded in bundle)
[Supabase PostgreSQL — RLS policies]
    ↓ fetch() calls
[FastAPI Serverless on Vercel — not reviewed]
```

> **WARNING**
> All authorization enforcement resides on the client side. The server-side (Supabase RLS, FastAPI) must be the authoritative enforcement layer, and currently the RLS policies do not enforce this.

## SECTION 4 — METHODOLOGY

### Standards Applied

- OWASP Top 10:2025 (primary classification)
- OWASP Web Security Testing Guide (WSTG) v4.2 (test methodology)
- OWASP Application Security Verification Standard (ASVS) v4.0 (verification requirements)
- CWE (Common Weakness Enumeration) for root cause classification
- CVSS 4.0 for severity scoring

### Testing Phases Executed

| Phase | Description | Status |
|---|---|---|
| 1 | Scope confirmation and authorization | ✅ Complete |
| 2 | Application reconnaissance and attack-surface mapping | ✅ Complete |
| 3 | Static source-code and configuration review | ✅ Complete |
| 4 | Dependency and supply chain analysis | ✅ Complete |
| 5 | Automated security testing (npm audit) | ✅ Complete |
| 6 | Controlled dynamic application testing | ❌ Not executed (no live URL) |
| 7 | Manual validation of potential vulnerabilities | ✅ Complete (code-level) |
| 8 | Risk analysis and evidence collection | ✅ Complete |
| 9 | Remediation recommendations | ✅ Produced |
| 10 | Regression testing and retesting | ⏳ Pending approval |
| 11 | Final report | ✅ This document |

### Tools Used

| Tool | Version | Purpose | Result |
|---|---|---|---|
| Static Code Review (Manual) | N/A | Full source read of all `.jsx`, `.js`, `.sql`, `.json`, `.yml` files | Completed |
| npm audit | Bundled w/ Node.js | Dependency vulnerability scan | 0 CVEs in 319 packages |
| Configuration Review | N/A | `vercel.json`, `vite.config.js`, `.gitignore`, CI/CD workflow | Completed |

## SECTION 5 — OWASP COVERAGE MATRIX

| OWASP ID | Category | Applicable | Tests Planned | Tests Executed | Tests Passed | Confirmed Findings | Potential Findings | Not Tested | S |
|---|---|---|---|---|---|---|---|---|---|
| A01:2025 | Broken Access Control | ✅ Yes | 12 | 9 | 6 | 2 | 1 | 3 | ⚠ F |
| A02:2025 | Security Misconfiguration | ✅ Yes | 14 | 11 | 7 | 2 | 1 | 3 | ⚠ F |
| A03:2025 | Software Supply Chain Failures | ✅ Yes | 12 | 8 | 8 | 0 | 1 | 4 | ✅ P |
| A04:2025 | Cryptographic Failures | ✅ Yes | 12 | 8 | 5 | 2 | 0 | 4 | ⚠ F |
| A05:2025 | Injection | ✅ Yes | 13 | 8 | 7 | 0 | 1 | 5 | ✅ P (p |
| A06:2025 | Insecure Design | ✅ Yes | 13 | 10 | 7 | 1 | 1 | 3 | ⚠ |

## SECTION 6 — FINDINGS SUMMARY

### Confirmed Vulnerabilities

| ID | Title | Severity | OWASP | Status |
|---|---|---|---|---|
| EPOTK-001 | Email-Based Admin Role Inference (Client-Side Escalation) | CRITICAL | A01, A07 | OPEN |
| EPOTK-002 | Production Secrets Committed to Repository | CRITICAL | A04, A02 | OPEN |
| EPOTK-003 | Supabase RLS Policies Grant Unauthenticated Full Data Access | HIGH | A01 | OPEN |
| EPOTK-004 | Authentication Bypass via Offline Demo Fallback | MEDIUM | A07, A06 | OPEN |
| EPOTK-005 | Missing HSTS and X-Frame-Options in Vercel Security Headers | MEDIUM | A02 | OPEN |
| EPOTK-006 | Security Events Not Logged (Auth Failures, Access Denials) | LOW | A09 | OPEN |

### Informational Observations

| ID | Title | Category |
|---|---|---|
| EPOTK-I01 | Hardcoded Admin Email in Navbar Component (`sriragavsurya@gmail.com`) | A02 / Privacy |
| EPOTK-I02 | `authConfirmed` Checkbox on Client Dashboard is Non-Binding | A06 / Insecure Design |

## SECTION 7 — DETAILED VULNERABILITY FINDINGS

**FINDING EPOTK-001**

**Title:** Email-Based Admin Role Inference — Client-Side Privilege Escalation

**Severity:** CRITICAL | **OWASP:** A01:2025, A07:2025

**CWE:** CWE-284, CWE-602 | **CVSS Score:** ~9.3 (Critical)

**Status:** OPEN

#### Affected Components

- `src/lib/auth.js` — Lines 1–9
- `src/pages/AuthPage.jsx` — Lines 16–26, 50–51
- `src/App.jsx` — Lines 22–28

#### Description

The application determines administrator privilege by checking whether the authenticated user's email address contains the substring `"admin"` — entirely on the client side. This logic is present in multiple locations.

**Location 1 — `auth.js`:**

```
export function getUserRole(user) {
  if (!user) return null;
  if (user.role) return user.role;
  const email = (user.email || '').toLowerCase();
  if (email.includes('admin')) return 'admin';   // ← VULNERABLE
  return 'client';
}
```

**Location 2 — `AuthPage.jsx`:**

```
const setDemoSession = (userEmail) => {
```

**Location 3 — `AuthPage.jsx` (post Supabase login):**

```
const userEmail = data.user.email?.toLowerCase() || '';
navigate(userEmail.includes('admin') ? '/admin' : '/dashboard');  // ← VULNERABLE
```

### Safe Reproduction Steps

1. Open the application login page ( `/auth` ).
2. Register a new account with any email containing `"admin"` — e.g., `notreallyadmin@attacker.com` or `xadmin@evil.com` .
3. Observe that after login, `getUserRole()` returns `'admin'` .
4. Navigate to `/admin` . The `ProtectedRoute` component passes because `role === 'admin'` .
5. Full admin dashboard, all client records, scan triggers, and delete capabilities are now accessible.

### Root Cause & Impact

**Root Cause:** Privilege assignment is performed exclusively client-side via string matching on a user-supplied value (email address), with no server-side verification at any API endpoint or database layer.

**Technical Impact:** Vertical privilege escalation to admin for any registered user. Attacker gains access to all service request records for all clients, can delete any record, change any status, and trigger target scans. If the FastAPI backend trusts the same session, impact extends to backend operations.

**Business Impact:** Complete compromise of all client data confidentiality, attacker impersonation of admin staff, and reputational damage.

### Recommended Remediation

**Step 1 — Remove email-based role inference entirely.** Replace `auth.js` :

```
export function getUserRole(user) {
  if (!user) return null;
  // Role must come from server-authoritative user_metadata
  return user?.user_metadata?.role || user?.app_metadata?.role || 'client';
}
```

**Step 2 — Set roles server-side using Supabase Admin API:**

```
const { data, error } = await supabaseAdmin.auth.admin.updateUserById(userId, {
  app_metadata: { role: 'admin' }
});
```

---

**FINDING EPOTK-002**

**Title:** Production Secrets Committed to Version-Controlled Repository

**Severity:** CRITICAL | **OWASP:** A04:2025, A02:2025

**CWE:** CWE-312, CWE-798

**Status:** OPEN

### Affected Components

- `.env` — Lines 1–5 (Supabase credentials)
- `.env.local` — Lines 1–6 (Supabase credentials + Vercel OIDC token)
- `.gitignore` — Lines 19–25 ( `.env` is listed but the file exists and was tracked)

### Description

The live Supabase project credentials ( `VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY` ) and a **Vercel OIDC token** are present in plaintext in `.env` and `.env.local` . While `.gitignore` includes `.env*` , these files exist on disk and were accessible during assessment. If the repository was ever committed with these files present, these secrets are in git history.

**Root Cause:** Environment files containing production credentials were committed to the repository. The Supabase anon key combined with overly permissive RLS creates a critical combined risk.

**Impact:** Any person with repository access can query the Supabase database. The Vercel OIDC token allows environment variable access and deployment compromise. Result is full exfiltration of `service_requests` and `security_audits` data.

### Recommended Remediation

1. **Immediately rotate the Supabase Anon Key** via the Supabase Dashboard.
2. **Revoke the Vercel OIDC token** and re-generate deployment credentials.
3. **Audit git history** and use `git filter-repo` to purge secrets from history if found.
4. **Never commit `.env` files** containing real values. Use Vercel Environment Variables exclusively for production.

---

**FINDING EPOTK-003**

**Title:** Supabase RLS Policies Grant Unauthenticated Users Full Data Access

**Severity:** HIGH | **OWASP:** A01:2025 | **CWE:** CWE-284, CWE-732

**Status:** OPEN

### Affected Component

`supabase_schema.sql` — Lines 59–90

### Description

The Row Level Security policies for both `service_requests` and `security_audits` tables grant SELECT, INSERT, and UPDATE to both `authenticated` and `anon` roles with `USING (true)` — meaning **any internet user, without logging in, can read all records and insert new ones**.

```
-- VULNERABLE — grants SELECT to anon role with no condition
CREATE POLICY "Allow select service_requests"
    ON public.service_requests
    FOR SELECT
    TO authenticated, anon     -- ← anon = unauthenticated internet user
    USING (true);              -- ← no filtering whatsoever
```

Any attacker who knows the Supabase URL and Anon Key (both committed to the repository) can perform unauthenticated reads of all client service requests using simple cURL commands.

### Root Cause & Impact

**Root Cause:** RLS policies were configured for maximum permissiveness. The `anon` role is used for unauthenticated requests.

**Impact:** Complete read access to all `service_requests` — exposing client names, target domains, priorities, and notes. Malicious records can be injected into the admin view.

### Recommended Remediation

**FINDING EPOTK-004**

**Title:** Authentication Bypass via Offline Demo Fallback

**Severity:** MEDIUM | **OWASP:** A07:2025, A06:2025

**Status:** OPEN

### Affected Component

`src/pages/AuthPage.jsx` — Lines 62–76, 79–82

### Description

The authentication flow contains paths that grant authenticated sessions without validating credentials with Supabase:

**Bypass 1 — Network error fallback:**

```
} catch (err) {
  if (errMsg.includes('Failed to fetch') || errMsg.includes('NetworkError') ...) {
    setTimeout(() => {
      setDemoSession(email);  // ← Grants session with unverified email
    }, 500);
  }
}
```

An attacker who can induce a network error receives a full authenticated session. The "Quick Login" buttons on the auth page also invoke `setDemoSession()` directly, bypassing authentication.

### Impact & Remediation

**Impact:** One-click unauthenticated admin access via the "Admin Persona" button present on the production login page.

**Remediation:** Remove the demo quick-login buttons from production builds. Remove the network-error fallback that grants sessions; show an error message instead.

---

**FINDING EPOTK-005**

**Title:** Missing HSTS and X-Frame-Options Headers in Vercel Configuration

**Severity:** MEDIUM | **OWASP:** A02:2025

**Status:** OPEN

### Affected Component

`vercel.json` — Lines 17–29

### Description

The `vercel.json` headers configuration includes CSP, `X-Content-Type-Options`, `Referrer-Policy`, and `Permissions-Policy` — but is missing `Strict-Transport-Security` (HSTS) and `X-Frame-Options`. Additionally, the CSP includes `script-src 'self' 'unsafe-inline'` which substantially weakens XSS protection.

### Recommended Remediation

Update `vercel.json` to include:

```
{
  "key": "Strict-Transport-Security",
  "value": "max-age=31536000; includeSubDomains; preload"
},
{
```

**FINDING EPOTK-006**

**Title:** No Security Event Logging for Authentication or Authorization Failures

**Severity:** LOW | **OWASP:** A09:2025 | **Status:** OPEN

**Description**

Authentication failures are logged only to `console.warn()` (developer-visible only). Authorization failures in `ProtectedRoute` are silently redirected with no logging whatsoever. No events are sent to Sentry or any backend logging service.

```
// App.jsx — unauthorized admin access attempt:
if (requireAdmin && role !== 'admin') {
  return ;  // ← silent redirect, no log
}
```

**Business Impact:** Brute-force attacks and unauthorized admin access attempts are completely invisible, leaving no audit trail for incident response.

**Remediation:** Integrate Sentry event capture for authentication failures ( `Sentry.captureEvent` ). Report unauthorized route access attempts to a backend logging endpoint.

---

### Informational Findings

**EPOTK-I01: Hardcoded Personal Email Address in Navbar Component**

A personal email address ( `sriragavsurya@gmail.com` ) is hardcoded as a constant `ADMIN_EMAIL` in `src/components/Navbar.jsx` . It will be included in the compiled bundle. **Recommendation:** Remove the unused constant.

**EPOTK-I02: `authConfirmed` Checkbox Is Non-Binding**

The client dashboard's scan form includes a checkbox for authorization confirmation. The state is sent in the POST body, but since the default is `true` and nothing prevents modifying this field, it provides no real authorization control. **Recommendation:** Ensure the FastAPI backend independently validates authorization.

## SECTION 8 — VERIFIED SECURITY CONTROLS

The following controls were inspected and verified as correctly implemented at code level:

| Control | Component | Verification Method | Result |
|---|---|---|---|
| Authenticated route guard | `App.jsx` | Code review | ✅ Present (client-side only) |
| Unauthenticated redirect | `App.jsx` | Code review | ✅ Redirects to `/auth` |
| Content-Security-Policy header | `vercel.json` | Configuration review | ⚠️ Present but `unsafe-inline` weakens it |
| X-Content-Type-Options | `vercel.json` | Configuration review | ✅ `nosniff` configured |
| Permissions-Policy | `vercel.json` | Configuration review | ✅ Restrictive policy configured |
| Referrer-Policy | `vercel.json` | Configuration review | ✅ `strict-origin-when-cross-origin` |
| Password field type | `AuthPage.jsx` | Code review | ✅ `type="password"` |

## SECTION 9 — REMEDIATION ROADMAP

| Priority | ID | Action | Files Affected | Verification |
|---|---|---|---|---|
| 🔴 P1 — Immediate | EPOTK-002 | Rotate Supabase Anon Key and Vercel OIDC token | Supabase Dashboard, Vercel | New key functional; old key rejected |
| 🔴 P1 — Immediate | EPOTK-001 | Remove email-based role inference; implement server-side role in `app_metadata` | `auth.js`, `AuthPage.jsx` | Non-admin email with 'admin' string cannot access `/admin` |
| 🔴 P1 — Immediate | EPOTK-003 | Replace permissive RLS policies with user-scoped policies | `supabase_schema.sql` | Anon key returns empty result set |
| 🟠 P2 — High | EPOTK-004 | Remove demo quick-login buttons; remove auth bypass fallbacks | `AuthPage.jsx` | Verified demo buttons absent from production |
| 🟡 P3 — Medium | EPOTK-005 | Add HSTS and X-Frame-Options headers; remove `unsafe-inline` from CSP | `vercel.json` | Headers present in HTTP response |
| 🔵 P4 — Low | EPOTK-006 | Integrate Sentry for auth/authz failure capture | `AuthPage.jsx`, `App.jsx` | Sentry events visible on failure |
| ℹ️ P5 — Info | EPOTK-I01 | Remove hardcoded `ADMIN_EMAIL` constant | `Navbar.jsx` | Constant absent from bundle |
| ℹ️ P5 — Info | EPOTK-I02 | Document `authConfirmed` is not a security control; enforce on backend | `DashboardPage.jsx` | Backend validates authorization |

## SECTION 10 — RETEST RESULTS

No fixes have been implemented yet. All findings are in **OPEN** status. This section will be updated after approved remediations are applied.

| ID | Retest Status |
|---|---|
| EPOTK-001 | OPEN — Awaiting approval |
| EPOTK-002 | OPEN — Awaiting approval |
| EPOTK-003 | OPEN — Awaiting approval |
| EPOTK-004 | OPEN — Awaiting approval |
| EPOTK-005 | OPEN — Awaiting approval |
| EPOTK-006 | OPEN — Awaiting approval |

## SECTION 11 — LIMITATIONS AND EXCLUSIONS

| Limitation | Impact |
|---|---|
| No live application URL provided | All dynamic tests (active HTTP, session manipulation, live API probing) were NOT EXECUTED |
| FastAPI backend source ( `/api/index.py` ) not present in repository | Backend injection, authorization, and error handling could not be reviewed |

## SECTION 12 — FINAL ASSESSMENT

### Observed Security Posture

The Epotech Cybersecurity Platform contains **two critical security vulnerabilities that, in combination, allow any internet user to gain full administrative access and read all client data without authentication**. The most severe finding (EPOTK-001 + EPOTK-002 + EPOTK-003 combined) represents a complete failure of the access control model. Production credentials are publicly accessible, the database enforces no meaningful access control for unauthenticated users, and administrative privilege can be obtained by anyone who registers with an email containing `"admin"`.

These findings are **not theoretical** — they are directly exploitable from source code inspection with no further analysis required.

### Next Steps

1. Obtain authorization to implement the remediation plan described in Section 9.
2. Execute fixes in P1 order: credential rotation → role system overhaul → RLS policy replacement.
3. Retest each finding after the fix is applied.
4. Conduct a follow-up dynamic assessment once a live URL is available.
5. Commission a review of the FastAPI backend source code.

> **IMPORTANT**
> This assessment does not certify the application as secure, compliant, or production-ready. It reflects the findings observable from static review on 2026-10-29. A dynamic follow-up assessment is required after remediation.

## SECTION 13 — TECHNICAL APPENDIX

### B. Key Source Files Reviewed

| File | Security-Relevant Content |
|---|---|
| `src/lib/auth.js` | Role inference logic (EPOTK-001) |
| `src/lib/supabase.js` | Supabase client initialization, credential usage |
| `src/lib/apiClient.js` / `api.js` | CRUD operations, fallback logic, Audit scan wrapper |
| `src/pages/AuthPage.jsx` | Authentication logic, demo bypass (EPOTK-004) |
| `src/App.jsx` | Route guards, ProtectedRoute component |
| `src/pages/AdminDashboard.jsx` | Admin panel — no server-side auth check |
| `src/pages/DashboardPage.jsx` | Client portal — scan triggers |
| `src/components/AdminRequestsTable.jsx` | CRUD operations on all requests |
| `src/components/Navbar.jsx` | Hardcoded admin email constant |
| `supabase_schema.sql` | RLS policy definitions (EPOTK-003) |
| `vercel.json` | HTTP security headers (EPOTK-005) |
| `.env` / `.env.local` | Credentials (EPOTK-002) |
| `package.json` / `package-lock.json` | Dependency manifests (0 known CVEs across 319 packages scanned via npm audit) |